STATE BOARD OF TECHNICAL EDUCATION KERALA

Directorate of Technical Education

Diploma in Computer Engineering
Semester I

LABORATORY MANUAL

1131 – PROBLEM SOLVING AND PYTHON PROGRAMMING

For Diploma Curriculum Revision 2026

Prepared by:
Department of Computer Science and Engineering

Academic Year: _____

PREFACE

This laboratory manual is prepared for first-semester diploma students studying *Problem Solving and Python Programming (1131)*. It follows the experiment sequence and course outcomes of the supplied 1131 laboratory syllabus. The format is based on the reference laboratory manual and includes the aim, objectives, theory, problem statements, algorithms where appropriate, Python programs, sample outputs, results, and practice questions.

Students should first understand the problem, identify the input and expected output, develop the logic, write the program, test it with suitable values, and record the actual output obtained in the laboratory.

COURSE OUTCOME MATRIX

CO	Course Outcome
CO1	Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs incorporating variables, fundamental data types, and standard input/output operations.
CO2	Optimize program execution through the strategic use of control structures and loop control statements.
CO3	Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations and built-in methods to solve programming problems.
CO4	Design and develop modular programs using functions, modules, and packages with proper scope management and argument-passing techniques.

GENERAL INSTRUCTIONS

1. Read the experiment before attending the laboratory session.
2. Write the experiment number, title, and date clearly in the record.
3. Understand the problem statement before writing the algorithm or program.
4. Type and run the program using the configured Python environment.
5. Check the output with more than one suitable input whenever possible.
6. Correct syntax and logical errors, and record the verified output.
7. Keep the record neat and submit it as instructed by the faculty.
8. Follow laboratory rules and use computers and software responsibly.

INDEX

Sl. No.	Title of Practical	Mapped CO	Page No.	Date
1	Algorithm and Flowchart	CO1		
2	Installation of Python and an IDE	CO1		
3	Introduction to Python Programming	CO1		
4	Decision Control Structures	CO2		
5	Selection-Based Control Structure	CO2		
6	Loop Control Structure	CO2		
7	String Manipulations	CO3		
8	List Operations	CO3		
9	Dictionary Operations	CO3		
10	Set Operations	CO3		
11	Built-in Functions	CO4		
12	User-Defined Functions	CO4		

EXPERIMENT NO. 1

ALGORITHM AND FLOWCHART FOR SIMPLE PROBLEMS

Date: _____

AIM

To understand algorithms and flowcharts by designing step-by-step solutions for simple problems before implementing them in Python.

OBJECTIVES

- Understand the purpose and characteristics of an algorithm.
- Identify common flowchart symbols.
- Develop a logical sequence of steps for a problem.
- Represent the steps using a flowchart.

THEORY

An **algorithm** is a finite sequence of clear steps used to solve a problem. A **flowchart** is a graphical representation of an algorithm using standard symbols connected by arrows. An oval usually represents Start/Stop, a parallelogram represents input/output, a rectangle represents processing, and a diamond represents a decision.

RELEVANT COURSE OUTCOME

CO1 – Design algorithms and flowcharts and develop foundational logical reasoning skills.

PROBLEM STATEMENTS

1. Calculate simple interest from principal, rate, and time.
2. Check whether a given integer is odd or even.
3. Display natural numbers from 1 to a given limit N.
4. Calculate an employee's salary from working days and daily wage.

ALGORITHM / FLOWCHART

Problem 1: Simple Interest

Formula: SI $= (P \times R \times T)/100$.

1. Start.
2. Input principal P, rate R, and time T.
3. Calculate SI $= (P \times R \times T)/100$.
4. Display SI.
5. Stop.

Problem 2: Odd or Even

1. Start; input integer N.
2. If $N \bmod 2 = 0$, display "Even Number".
3. Otherwise, display "Odd Number".
4. Stop.

Problem 3: Natural Numbers up to N

1. Start; input N; set $i = 1$.
2. While $i \leq N$, display i and update $i = i + 1$.
3. Stop.

Problem 4: Employee Salary

Formula: Salary = WorkingDays $\times$ DailyWage.

1. Start; input working days and daily wage.
2. Calculate salary.
3. Display salary and stop.

FLOWCHART RECORD

Draw a separate flowchart for each problem using standard symbols. Connect the symbols with arrows and label the decision branches Yes/No where required.

RESULT

Algorithms for simple interest, odd/even checking, printing natural numbers, and employee salary calculation were designed. The corresponding flowcharts should be drawn and verified in the record.

PRACTICAL RELATED QUESTIONS

1. What is an algorithm?
2. What is a flowchart?
3. Which flowchart symbol represents a decision?
4. Why should program logic be planned before coding?

EXPERIMENT NO. 2

INSTALLATION OF PYTHON AND AN IDE

Date: _____

AIM

To install or verify the Python interpreter and configure an Integrated Development Environment (IDE) for writing and executing Python programs.

OBJECTIVES

- Understand the Python interpreter and IDE.
- Verify Python installation.
- Create and execute a Python file.

THEORY

The Python interpreter runs Python code. An IDE provides tools such as a code editor, program execution, syntax highlighting, and debugging. Visual Studio Code (VS Code) can be used for Python programming after installing the Python extension.

RESOURCES REQUIRED

A computer with Windows or Ubuntu, internet access for installation, Python, and an IDE such as VS Code.

PROCEDURE

A. Verify or install Python in Windows

1. Visit https://www.python.org/downloads/ and download the Windows installer.
2. Run the installer and enable "Add Python to PATH" when that option is available.
3. Complete installation.
4. Open Command Prompt and run `python -version`.

B. Verify or install Python in Ubuntu

1. Open Terminal.
2. If Python 3 is not installed, use the package manager as directed by the laboratory instructor.
3. Verify with `python3 -version`.

C. Configure VS Code

1. Download VS Code from https://code.visualstudio.com/.
2. Install and open VS Code.
3. Open Extensions, search for Python, and install the Python extension.
4. Create a file named `hello.py` and enter the following code.

```
1 print("Hello, Python!")
```

Run the file using the IDE or the appropriate terminal command for the operating system.

SAMPLE OUTPUT

```
Hello, Python!
```

RESULT

Python and the selected IDE were verified or configured, and a simple Python program was executed. Record the version and output obtained on the laboratory computer.

PRACTICAL RELATED QUESTIONS

1. What is a Python interpreter?
2. What is an IDE?
3. What is the purpose of the Python extension in VS Code?
4. What is the extension of a Python source file?

EXPERIMENT NO. 3

INTRODUCTION TO PYTHON PROGRAMMING

Date: _____

AIM

To write and execute basic Python programs using variables, input/output statements, data types, and arithmetic operators.

OBJECTIVES

- Use variables to store values.
- Accept input using `input()` and display results using `print()`.
- Perform arithmetic calculations.
- Convert input strings to numeric values when necessary.

RELEVANT COURSE OUTCOME

CO1 – Develop Python programs using variables, fundamental data types, and standard input/output operations.

THEORY

Python is a high-level, general-purpose programming language. Variables refer to stored values. The `input()` function reads user input as text; `int()` and `float()` convert input to numeric types. The `print()` function displays information. Arithmetic operators include `+`, `-`, `*`, `/`, `//`, `%`, and `**`.

PROBLEM 1: TEMPERATURE CONVERSION

Convert temperature from Celsius to Fahrenheit using $F = (C \times 9/5) + 32$.

Algorithm: Start; input Celsius C; calculate $F = (C \times 9/5) + 32$; display F; stop.

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print("Temperature in Fahrenheit:", fahrenheit)
```

SAMPLE OUTPUT

```
Enter temperature in Celsius: 30
Temperature in Fahrenheit: 86.0
```

PROBLEM 2: EMPLOYEE SALARY

Calculate salary using working days multiplied by daily wage.

```python
working_days = int(input("Enter number of working days: "))
daily_wage = float(input("Enter daily wage: "))
salary = working_days * daily_wage
print("Monthly Salary:", salary)
```

SAMPLE OUTPUT

```
Enter number of working days: 26
Enter daily wage: 850
Monthly Salary: 22100.0
```

PROBLEM 3: STUDENT PERCENTAGE

Calculate total and percentage for five subjects, assuming each subject is out of 100.

```python
m1 = float(input("Enter marks of subject 1: "))
m2 = float(input("Enter marks of subject 2: "))
m3 = float(input("Enter marks of subject 3: "))
m4 = float(input("Enter marks of subject 4: "))
m5 = float(input("Enter marks of subject 5: "))
total = m1 + m2 + m3 + m4 + m5
percentage = (total / 500) * 100
print("Total Marks:", total)
print("Percentage:", percentage)
```

SAMPLE OUTPUT

```
Enter marks of subject 1: 80
Enter marks of subject 2: 75
Enter marks of subject 3: 90
Enter marks of subject 4: 85
Enter marks of subject 5: 70
Total Marks: 400.0
Percentage: 80.0
```

RESULT

The programs for temperature conversion, employee salary, and student percentage were executed and their outputs verified.

PRACTICAL RELATED QUESTIONS

1. What is a variable?
2. What is the difference between `int()` and `float()`?
3. What is the purpose of `input()`?
4. What does the modulus operator return?

EXPERIMENT NO. 4

DECISION CONTROL STRUCTURES

Date: _____

AIM

To develop Python programs using conditional statements to make decisions based on input values.

OBJECTIVES

Understand `if`, `if-else`, and `if-elif-else`; use relational and logical operators; test programs using different inputs.

THEORY

A decision statement selects which block of code runs according to a condition. Python uses indentation to define the body of a conditional statement.

RELEVANT COURSE OUTCOME

CO2 – Use control structures to solve programming problems.

PROBLEM 1: BUS TICKET CHARGE

For this exercise, the supplied reference rule is: if the passenger is female or older than 60, set the charge to zero; otherwise retain the entered charge.

```
gender = input("Enter gender (male/female): ")
age = int(input("Enter age: "))
charge = float(input("Enter bus charge: "))

if gender.lower() == "female" or age > 60:
    charge = 0

print("Bus Ticket Charge:", charge)
```

SAMPLE OUTPUT

```
Enter gender (male/female): female
Enter age: 35
Enter bus charge: 50
Bus Ticket Charge: 0
```

PROBLEM 2: LEAP YEAR CHECKING

A year is a leap year if it is divisible by 400, or divisible by 4 but not by 100.

```python
year = int(input("Enter a year: "))

if year % 400 == 0:
    print("Leap Year")
elif year % 100 == 0:
    print("Not a Leap Year")
elif year % 4 == 0:
    print("Leap Year")
else:
    print("Not a Leap Year")
```

SAMPLE OUTPUT

```
Enter a year: 2024
Leap Year
```

PROBLEM 3: ELECTRICITY BILL

Use the rates specified in the reference manual: up to 100 units at Rs. 5 per unit, above 100 and up to 300 at Rs. 8 per unit, and above 300 at Rs. 10 per unit. The program below applies the selected rate to all units consumed.

```python
units = int(input("Enter electricity units consumed: "))

if units <= 100:
    bill = units * 5
elif units <= 300:
    bill = units * 8
else:
    bill = units * 10

print("Electricity Bill Amount: Rs.", bill)
```

SAMPLE OUTPUT

```
Enter electricity units consumed: 250
```

```
Electricity Bill Amount: Rs. 2000
```

RESULT

The programs using conditional statements were executed and tested.

PRACTICAL RELATED QUESTIONS

1. What is the purpose of an `if` statement?
2. What is the difference between `and` and `or`?
3. Why is the leap-year condition checked in a particular order?
4. What is the role of indentation in Python?

EXPERIMENT NO. 5

SELECTION-BASED CONTROL STRUCTURE

Date: _____

AIM

To write Python programs using an if-elif-else ladder and match-case statement for selecting one outcome from multiple alternatives.

THEORY

An `if-elif-else` ladder checks conditions in order. The `match-case` statement selects a case based on the value or pattern being matched and is available in Python 3.10 and later.

RELEVANT COURSE OUTCOME

CO2 – Apply control structures to select the appropriate action.

PROBLEM 1: STUDENT GRADE

Use the following grading policy: 90 and above: A; 80–89: B; 70–79: C; 60–69: D; below 60: Fail.

```
mark = int(input("Enter student's mark: "))

if mark >= 90:
    grade = "A"
elif mark >= 80:
    grade = "B"
elif mark >= 70:
    grade = "C"
elif mark >= 60:
    grade = "D"
else:
    grade = "Fail"

print("Grade:", grade)
```

SAMPLE OUTPUT

```
Enter student's mark: 85
Grade: B
```

PROBLEM 2: DAY NAME USING MATCH-CASE

```python
day = int(input("Enter day number (1-7): "))

match day:
    case 1:
        print("Sunday")
    case 2:
        print("Monday")
    case 3:
        print("Tuesday")
    case 4:
        print("Wednesday")
    case 5:
        print("Thursday")
    case 6:
        print("Friday")
    case 7:
        print("Saturday")
    case _:
        print("Invalid Day Number")
```

SAMPLE OUTPUT

```
Enter day number (1-7): 4
Wednesday
```

RESULT

The grade evaluation and day-name selection programs were executed and tested with valid and invalid inputs.

PRACTICAL RELATED QUESTIONS

1. When is an if-elif-else ladder useful?
2. What does the underscore case `case _` represent?
3. Which Python version introduced `match-case`?

EXPERIMENT NO. 6

LOOP CONTROL STRUCTURE

Date: _____

AIM

To write Python programs using loops to repeat operations and solve problems involving sequences and repeated calculations.

THEORY

A loop executes a block of code repeatedly. A `for` loop is commonly used to iterate over a sequence or range. A `while` loop repeats while its condition remains true. The `break` statement exits a loop, while `continue` skips to the next iteration.

RELEVANT COURSE OUTCOME

CO2 – Use loops and loop-control statements effectively.

PROBLEM 1: PRINT NATURAL NUMBERS UP TO N

```
n = int(input("Enter the limit: "))

for i in range(1, n + 1):
    print(i, end=" ")
print()
```

SAMPLE OUTPUT

```
Enter the limit: 5
1 2 3 4 5
```

PROBLEM 2: SUM OF FIRST N NATURAL NUMBERS

```python
n = int(input("Enter the limit: "))
total = 0

for i in range(1, n + 1):
    total = total + i

print("Sum:", total)
```

SAMPLE OUTPUT

```
Enter the limit: 5
Sum: 15
```

PROBLEM 3: COUNT VOWELS IN A WORD

```python
word = input("Enter a word: ")
count = 0

for ch in word.lower():
    if ch in "aeiou":
        count += 1

print("Number of vowels:", count)
```

SAMPLE OUTPUT

```
Enter a word: Education
Number of vowels: 5
```

RESULT

Loop-based programs were executed and checked using suitable input values.

PRACTICAL RELATED QUESTIONS

1. State one difference between `for` and `while`.
2. What is an infinite loop?
3. What does `break` do?
4. Why must a loop's condition or control variable be updated when required?

EXPERIMENT NO. 7

STRING MANIPULATIONS

Date: _____

AIM

To perform string operations such as indexing, slicing, concatenation, membership testing, searching, replacing, and formatting.

THEORY

A string is a sequence of characters enclosed in quotes. Strings are immutable: string methods produce new strings rather than changing the original string in place. Indexing accesses individual characters, slicing extracts a portion, and methods such as `upper()`, `lower()`, `replace()`, `find()`, `count()`, `strip()`, and `split()` help process text.

RELEVANT COURSE OUTCOME

CO3 – Manage string data using operations and built-in methods.

PROBLEM STATEMENT

Develop a customer feedback program that accepts customer details and performs common string operations.

```
name = input("Enter customer name: ").strip()
customer_id = input("Enter customer ID: ").strip()
email = input("Enter email address: ").strip()
feedback = input("Enter feedback: ").strip()

print("First character:", name[0] if name else "")
print("Last character:", name[-1] if name else "")
print("First five feedback characters:", feedback[:5])
print("Last five feedback characters:", feedback[-5:])

first_name = name.split()[0] if name else ""
username = first_name + customer_id
print("Username:", username)
print("Contains 'good':", "good" in feedback.lower())
```

```
print("Uppercase:", feedback.upper())
print("Lowercase:", feedback.lower())
print("Capitalized:", feedback.capitalize())
print("Updated feedback:", feedback.lower().replace("bad", "improve"))
print("Position of 'delivery':", feedback.lower().find("delivery"))
print("Count of 'good':", feedback.lower().count("good"))
print("Words:", feedback.split())
print("Email contains @:", "@" in email)
```

SAMPLE OUTPUT

```
Enter customer name: Anu
Enter customer ID: 102
Enter email address: anu@example.com
Enter feedback: Good delivery and good service
First character: A
Last character: u
First five feedback characters: Good
Last five feedback characters: rvice
Username: Anu102
Contains 'good': True
Uppercase: GOOD DELIVERY AND GOOD SERVICE
Lowercase: good delivery and good service
Capitalized: Good delivery and good service
Updated feedback: good delivery and good service
Position of 'delivery': 5
Count of 'good': 2
Words: ['Good', 'delivery', 'and', 'good', 'service']
Email contains @: True
```

RESULT

String indexing, slicing, joining, membership checks, and common string methods were demonstrated.

PRACTICAL RELATED QUESTIONS

1. What is a string?
2. What is meant by string immutability?
3. What is the difference between indexing and slicing?
4. What do `strip()` and `split()` do?

EXPERIMENT NO. 8

LIST OPERATIONS

Date: _____

AIM

To create and manipulate lists using indexing, insertion, deletion, membership testing, slicing, sorting, and list methods.

THEORY

A list stores an ordered collection of items and is written using square brackets. Lists are mutable, so elements can be added, removed, or changed. Common operations include `append()`, `insert()`, `remove()`, `len()`, `sort()`, `reverse()`, `count()`, and `index()`.

RELEVANT COURSE OUTCOME

CO3 – Manage sequential data using list operations and built-in methods.

PROBLEM STATEMENT

Create a product inventory and demonstrate list operations.

```
products = ["Rice", "Milk", "Bread", "Tea"]
print("Initial list:", products)
print("First product:", products[0])
print("Last product:", products[-1])

products.append("Sugar")
products.insert(1, "Coffee")
print("After adding products:", products)

products.remove("Bread")
print("After removing Bread:", products)
print("Number of products:", len(products))
print("Is Milk available?", "Milk" in products)
print("Slice [0:3]:", products[0:3])

products.sort()
```

```
print("Sorted list:", products)
products.reverse()
print("Reversed list:", products)
print("Count of Rice:", products.count("Rice"))
print("Index of Milk:", products.index("Milk"))
print("Final inventory:", products)
```

SAMPLE OUTPUT

```
Initial list: ['Rice', 'Milk', 'Bread', 'Tea']
First product: Rice
Last product: Tea
After adding products: ['Rice', 'Coffee', 'Milk', 'Bread', 'Tea', 'Sugar']
After removing Bread: ['Rice', 'Coffee', 'Milk', 'Tea', 'Sugar']
Number of products: 5
Is Milk available? True
Slice [0:3]: ['Rice', 'Coffee', 'Milk']
Sorted list: ['Coffee', 'Milk', 'Rice', 'Sugar', 'Tea']
Reversed list: ['Tea', 'Sugar', 'Rice', 'Milk', 'Coffee']
Count of Rice: 1
Index of Milk: 3
Final inventory: ['Tea', 'Sugar', 'Rice', 'Milk', 'Coffee']
```

RESULT

The list was created and updated using common list operations.

PRACTICAL RELATED QUESTIONS

1. How is a list created?
2. How does `append()` differ from `insert()`?
3. What does `len()` return?
4. Are lists mutable?

EXPERIMENT NO. 9

DICTIONARY OPERATIONS

Date: _____

AIM

To store and manage data as key-value pairs using Python dictionaries.

THEORY

A dictionary stores information as key-value pairs and is written using curly braces. Keys must be unique. Dictionaries are mutable, so entries can be added, updated, or removed. Common methods include `keys()`, `values()`, `items()`, and `pop()`.

RELEVANT COURSE OUTCOME

CO3 – Use dictionaries to organize and retrieve structured data.

PROBLEM STATEMENT

Create a product dictionary using product IDs as keys and product names as values.

```
products = {101: "Rice", 102: "Milk", 103: "Tea"}
print("Initial dictionary:", products)
print("Product 102:", products[102])

products[104] = "Sugar"
products[103] = "Green Tea"
print("After add/update:", products)

products.pop(101)
print("After removing product 101:", products)
print("Product IDs:", list(products.keys()))
print("Product names:", list(products.values()))
print("Key-value pairs:", list(products.items()))
print("Is product 102 present?", 102 in products)
print("Number of products:", len(products))
print("Final dictionary:", products)
```

SAMPLE OUTPUT

```
Initial dictionary: {101: 'Rice', 102: 'Milk', 103: 'Tea'}
Product 102: Milk
After add/update: {101: 'Rice', 102: 'Milk', 103: 'Green Tea', 104: 'Sugar'}
After removing product 101: {102: 'Milk', 103: 'Green Tea', 104: 'Sugar'}
Product IDs: [102, 103, 104]
Product names: ['Milk', 'Green Tea', 'Sugar']
Key-value pairs: [(102, 'Milk'), (103, 'Green Tea'), (104, 'Sugar')]
Is product 102 present? True
Number of products: 3
Final dictionary: {102: 'Milk', 103: 'Green Tea', 104: 'Sugar'}
```

RESULT

Dictionary entries were accessed, added, updated, removed, and displayed using dictionary methods.

PRACTICAL RELATED QUESTIONS

1. What is a key-value pair?
2. Can a dictionary contain duplicate keys?
3. What is the purpose of `items()`?
4. What happens when an existing key is assigned a new value?

EXPERIMENT NO. 10

SET OPERATIONS

Date: _____

AIM

To create sets and perform common operations including adding, removing, membership testing, union, intersection, and difference.

THEORY

A set is an unordered collection of distinct elements. Duplicate values are stored only once. Sets are useful for membership checks and comparing groups of values. The operators `|`, `&`, and `-` represent union, intersection, and difference, respectively.

RELEVANT COURSE OUTCOME

CO3 – Apply set operations to manage collections of distinct values.

PROBLEM STATEMENT

Maintain library members from two branches and compare their membership lists.

```
branch_a = {"Anu", "Binu", "Ciya", "Dev"}
branch_b = {"Ciya", "Dev", "Esha"}

print("Branch A:", branch_a)
branch_a.add("Fathima")
branch_a.discard("Binu")
print("Updated Branch A:", branch_a)
print("Is Anu registered?", "Anu" in branch_a)
print("Number of members in Branch A:", len(branch_a))

print("Union:", branch_a | branch_b)
print("Intersection:", branch_a & branch_b)
print("Members only in Branch A:", branch_a - branch_b)
```

SAMPLE OUTPUT

```
Branch A: {'Anu', 'Binu', 'Ciya', 'Dev'}
Updated Branch A: {'Anu', 'Ciya', 'Dev', 'Fathima'}
Is Anu registered? True
Number of members in Branch A: 4
Union: {'Anu', 'Ciya', 'Dev', 'Esha', 'Fathima'}
Intersection: {'Ciya', 'Dev'}
Members only in Branch A: {'Anu', 'Fathima'}
```

Note: The order in which set elements are displayed may differ between runs.

RESULT

Set membership and the union, intersection, and difference operations were demonstrated.

PRACTICAL RELATED QUESTIONS

1. How is a set different from a list?
2. Why are duplicate elements not retained in a set?
3. What is the difference between union and intersection?
4. What does the expression $A - B$ represent?

EXPERIMENT NO. 11

BUILT-IN FUNCTIONS

Date: _____

AIM

To use suitable Python built-in functions to process values and collections.

THEORY

Built-in functions are available in Python without defining them first. Examples include `len()`, `sum()`, `min()`, `max()`, `sorted()`, `type()`, `abs()`, and numeric conversion functions such as `int()` and `float()`. Select a function based on the operation required.

RELEVANT COURSE OUTCOME

CO4 – Use functions and built-in facilities in Python programs.

PROBLEM STATEMENT

Read marks, then use built-in functions to report basic statistics.

```
marks = [78, 85, 92, 67, 88]

print("Marks:", marks)
print("Number of marks:", len(marks))
print("Total:", sum(marks))
print("Highest mark:", max(marks))
print("Lowest mark:", min(marks))
print("Sorted marks:", sorted(marks))
print("Absolute value of -12:", abs(-12))
print("Data type of marks:", type(marks))
average = sum(marks) / len(marks)
print("Average:", average)
```

SAMPLE OUTPUT

```
Marks: [78, 85, 92, 67, 88]
Number of marks: 5
Total: 410
Highest mark: 92
Lowest mark: 67
Sorted marks: [67, 78, 85, 88, 92]
Absolute value of -12: 12
Data type of marks: <class 'list'>
Average: 82.0
```

RESULT

Built-in functions were used to find collection length, total, minimum, maximum, sorted values, absolute value, and data type.

PRACTICAL RELATED QUESTIONS

1. What does `len()` return?
2. What is the difference between `sort()` and `sorted()`?
3. What does `type()` return?
4. How can an average be calculated using `sum()` and `len()`?

EXPERIMENT NO. 12

USER-DEFINED FUNCTIONS

Date: _____

AIM

To define and call functions in Python and use parameters, return values, and local variables to create modular programs.

THEORY

A user-defined function is a named block of reusable code created using the `def` keyword. Parameters receive values when the function is called. The `return` statement sends a result back to the caller. Functions help divide a large problem into smaller, understandable parts and reduce repeated code.

RELEVANT COURSE OUTCOME

CO4 – Develop modular programs using functions, scope management, and argument passing.

PROBLEM 1: CALCULATE SIMPLE INTEREST USING A FUNCTION

```
def simple_interest(principal, rate, time):
    interest = (principal * rate * time) / 100
    return interest

p = float(input("Enter principal: "))
r = float(input("Enter rate of interest: "))
t = float(input("Enter time in years: "))

answer = simple_interest(p, r, t)
print("Simple Interest:", answer)
```

SAMPLE OUTPUT

```
Enter principal: 10000
Enter rate of interest: 5
Enter time in years: 2
Simple Interest: 1000.0
```

PROBLEM 2: CHECK EVEN OR ODD USING A FUNCTION

```python
def even_or_odd(number):
    if number % 2 == 0:
        return "Even"
    return "Odd"

n = int(input("Enter an integer: "))
print(n, "is", even_or_odd(n))
```

SAMPLE OUTPUT

```
Enter an integer: 7
7 is Odd
```

PROBLEM 3: FUNCTION WITH DEFAULT ARGUMENT

```python
def greet(name="Student"):
    print("Welcome,", name)

greet()
greet("Anu")
```

SAMPLE OUTPUT

```
Welcome, Student
Welcome, Anu
```

RESULT

User-defined functions were created and called. Parameters, return values, and a default argument were demonstrated.

PRACTICAL RELATED QUESTIONS

1. Which keyword is used to define a function?
2. What is the purpose of `return`?
3. What is a parameter?
4. What is a default argument?
5. What is the difference between a local variable and a global variable?

ADDITIONAL PRACTICE

1. Write a program to find the largest of three numbers.
2. Write a program to calculate factorial using a loop.
3. Accept a sentence and count its words.
4. Store five student names in a list and search for a given name.
5. Create a dictionary of student names and marks, then update one student's mark.
6. Create two sets of course participants and display common participants.
7. Write a function that returns the square of a number.
8. Write a function to calculate the average of a list of numbers.

REFERENCES

1. *1131 – Problem Solving and Python Programming Laboratory Manual*, Diploma Curriculum Revision 2026, State Institute of Technical Teachers Training and Research, Kalamassery.
2. Python documentation: https://docs.python.org/3/
3. Python downloads: https://www.python.org/downloads/
4. Visual Studio Code: https://code.visualstudio.com/